

PROYECTO FINAL · MAESTRÍA EN ARQUITECTURA DE SOFTWARE

Arquitectura híbrida de base de datos para comercio electrónico
PostgreSQL / Supabase + MongoDB Atlas

Giovani Esteban Romero Hurtado

Universidad de La Sabana — Maestría en Arquitectura de Software

Por qué Ecommify necesita una base de datos híbrida



Problema de negocio

Ecommify —plataforma de comercio electrónico tecnológico— debe procesar pedidos, pagos e inventario con alta consistencia, y a la vez explotar catálogos, reseñas y comportamiento de usuario para analítica. Un único motor relacional no equilibra ambos patrones de uso.



Objetivo principal

Diseñar una arquitectura híbrida que gestione datos transaccionales y analíticos garantizando consistencia, escalabilidad, disponibilidad, rendimiento y flexibilidad.

ALCANCE DEL PROYECTO

- Esquema relacional completo en PostgreSQL
- Colecciones documentales en MongoDB
- Carga de los 9 datasets de Olist (~1,5M registros)
- Construcción de consultas críticas y análisis con EXPLAIN
- Diseño de índices y particionamiento mensual
- Validación de rendimiento y documentación de resultados

Dos motores, responsabilidades diferenciadas



PostgreSQL / Supabase

Núcleo transaccional y relacional

- Clientes, pedidos, pagos, productos, vendedores, ítems
- Integridad referencial y transacciones ACID
- Índices especializados (B-tree, parciales, GIN)
- Particionamiento RANGE mensual de orders



MongoDB Atlas

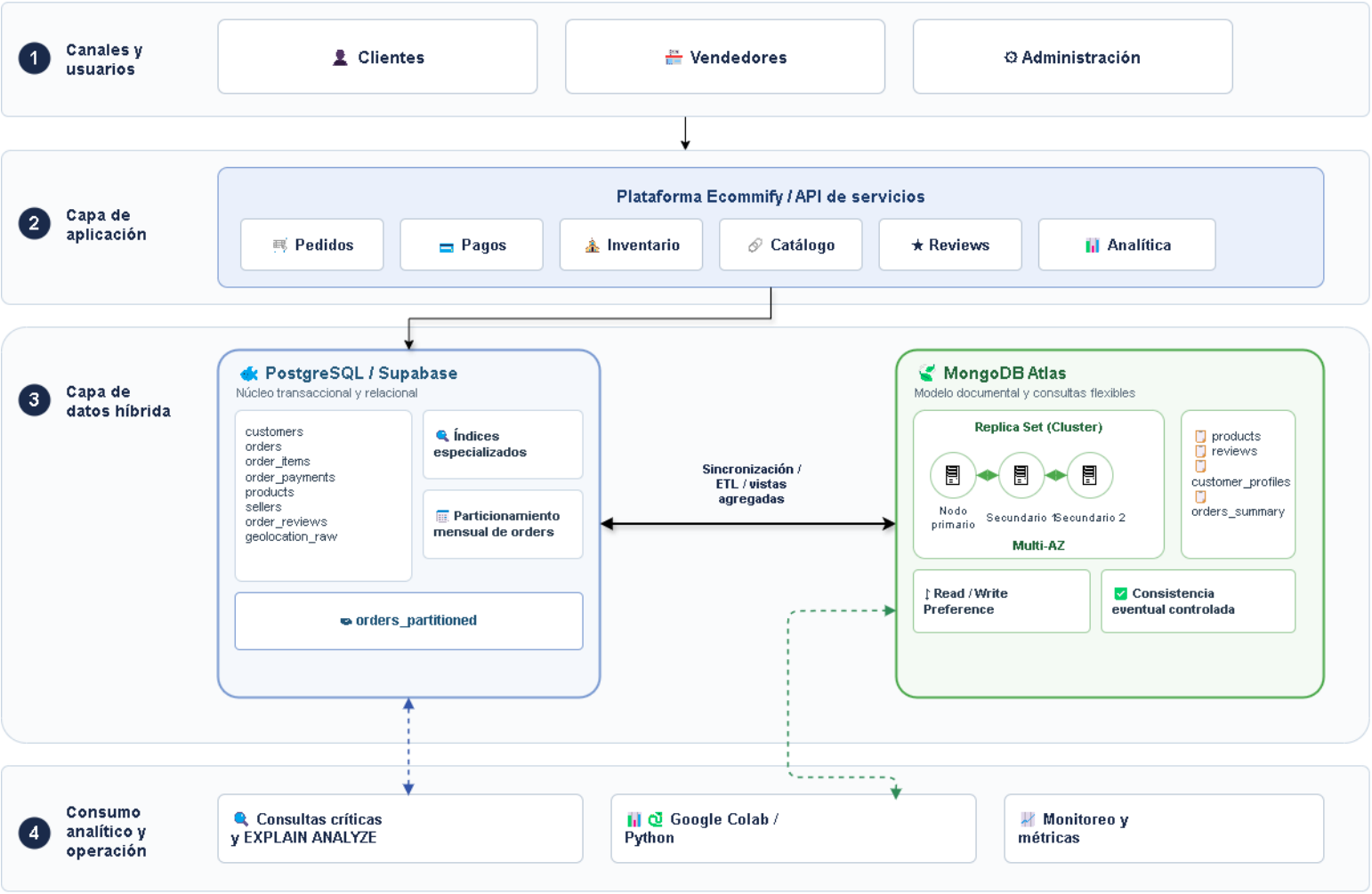
Capa documental complementaria

- Reviews, catálogo extendido, perfiles, orders_summary
- Patrones embedding / referencing según consulta
- Índices simples, compuestos y de texto
- Réplica set multi-AZ con 3 nodos

Sincronización ETL entre motores: vistas agregadas, validaciones, idempotencia y reintentos controlados.

B. ARQUITECTURA HÍBRIDA IMPLEMENTADA

Arquitectura híbrida de datos - Ecommify



Qué va en cada motor, y por qué

Dato / dominio	Motor	Justificación técnica
Pedidos, pagos, inventario	PostgreSQL	Consistencia fuerte, transacciones ACID, integridad referencial
Clientes, ítems de pedido, vendedores	PostgreSQL	Joins relacionales, trazabilidad y particionamiento por fecha
Reseñas enriquecidas y catálogo extendido	MongoDB	Texto libre, atributos variables por categoría, búsqueda flexible
Perfiles consolidados y comportamiento	MongoDB	Alta volumetría semiestructurada, lectura rápida sin joins
Documentos agregados de pedidos	MongoDB	Vistas de lectura tipo "Order 360" sin múltiples uniones

Optimización en PostgreSQL: antes vs. después

87,77%

mejora en tiempo total
de ejecución

2.786,066 ms → 340,828 ms

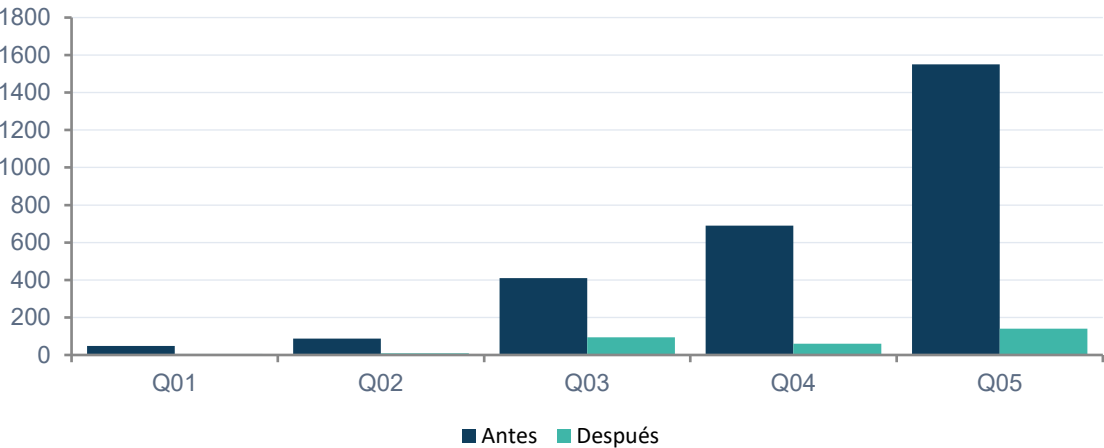
88,85%

reducción de bloques
leídos

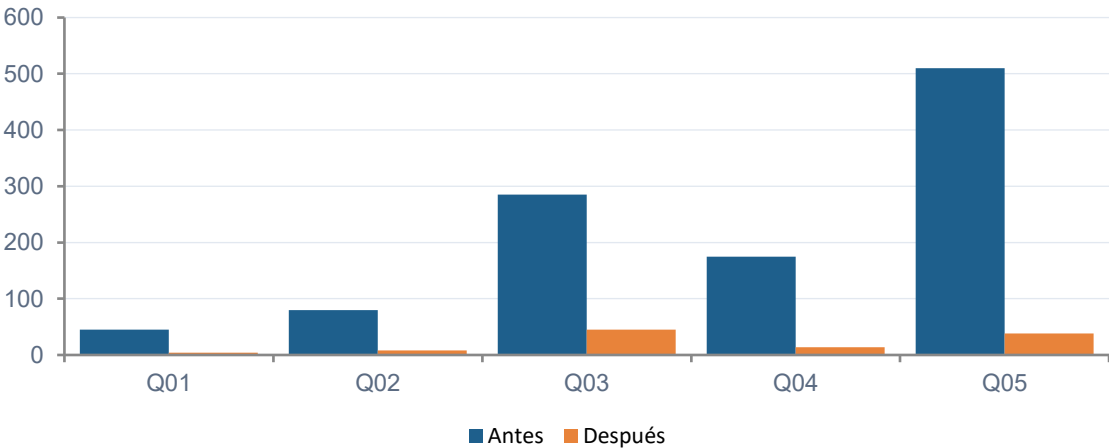
1.103 → 123 bloques

Aplicado mediante índices especializados (B-tree, parciales, GIN) y particionamiento RANGE mensual sobre la tabla orders, validado con EXPLAIN (ANALYZE, BUFFERS).

Tiempo de ejecución por consulta (ms)



Bloques leídos por consulta



Cada motor gana donde aporta su fortaleza

PostgreSQL — gana en

- Consistencia transaccional y ACID
- Integridad referencial (FK, constraints)
- Consultas relacionales complejas (JOIN, agregaciones SQL)
- Rendimiento optimizado: -87,77% tiempo, -88,85% bloques
- Análisis histórico con particionamiento mensual
- Control financiero y conciliación de pagos

MongoDB — gana en

- Datos documentales y atributos flexibles
- Escalabilidad documental sin joins múltiples
- Disponibilidad y resiliencia (réplica set multi-AZ)
- Búsqueda textual y datos semiestructurados
- Catálogo enriquecido con especificaciones variables
- Tolerancia a consistencia eventual (reseñas, eventos)

Prioridad CAP por módulo de negocio

Módulo	Motor	CAP	Justificación
Pedidos	PostgreSQL	CP	Consistencia en creación, estado y trazabilidad del pedido
Pagos	PostgreSQL	CP	Integridad financiera; evita duplicidades y errores de conciliación
Inventario	PostgreSQL	CP	Evita sobreventa por datos de stock inconsistentes
Catálogo extendido	MongoDB	AP	Tolera consistencia eventual; prioriza disponibilidad y flexibilidad
Reseñas enriquecidas	MongoDB	AP	No requiere consistencia inmediata; replicación eventual
Comportamiento usuarios	MongoDB	AP	Captura de eventos con alta disponibilidad, proceso posterior
Sincronización ETL	Capa intermedia	CP controlado	Checkpoints, reintentos e idempotencia entre motores

CP: prioriza Consistencia y tolerancia a fallos de red · AP: prioriza Disponibilidad y tolerancia a fallos de red

Qué aprendimos al diseñar la arquitectura híbrida



No todo dato requiere el mismo motor

PostgreSQL para transacciones críticas; MongoDB para documentos flexibles y agregados de lectura.



La consistencia se prioriza por criticidad

Pedidos, pagos e inventario permanecen en PostgreSQL para minimizar riesgo operativo y financiero.



El rendimiento depende del diseño físico

Índices, particionamiento y planes de ejecución explican la mejora del 87,77% — no solo la tecnología.



La sincronización es un punto crítico

El flujo ETL PostgreSQL → MongoDB exige checkpoints, reintentos e idempotencia para evitar inconsistencias.



CAP debe analizarse por módulo

No existe una única respuesta: CP para transacciones, AP para lectura documental, según el caso de uso.



El free tier limita las pruebas reales

La carga controlada e índices selectivos permitieron validar la arquitectura sin sobredimensionar costos.

Cómo sostener y escalar la arquitectura híbrida



Mantener la división de motores

PostgreSQL como motor transaccional principal para integridad y trazabilidad; MongoDB para escenarios documentales, catálogos enriquecidos y lectura flexible — evitando sobrecargar un único motor con responsabilidades incompatibles.



Disciplina de rendimiento continua

Índices selectivos, revisión periódica con EXPLAIN (ANALYZE, BUFFERS), automatización de particiones futuras y control de la partición DEFAULT para evitar acumulación fuera de rango.

ESCALAMIENTO Y TECNOLOGÍA COMPLEMENTARIA

Plan 10x: Particionar `order_items`, `order_payments` y `geolocation_raw`; automatizar particiones con `pg_cron`; evaluar sharding en MongoDB.

Caching: Redis/Valkey para catálogo y sesiones; CDN para imágenes — nunca para pagos o datos sensibles.

Search: OpenSearch o MongoDB Atlas Search para búsqueda avanzada de productos y reseñas.

Observabilidad: OpenTelemetry, Prometheus/Grafana y monitoreo de replication lag end-to-end.

Migración a producción: Clúster dedicado multi-AZ, backups automáticos, roles, secretos y ambientes Dev/QA/Prod.

I. CONCLUSIONES

Cumplimiento de objetivos del proyecto



Diseñar una arquitectura híbrida PostgreSQL/MongoDB

Separación entre núcleo transaccional relacional y capa documental flexible



Implementar PostgreSQL como base transaccional

Modelo relacional con integridad, consistencia, índices y particionamiento



Implementar MongoDB como capa documental

Colecciones para catálogo extendido, reseñas, perfiles y agregados



Optimizar el rendimiento de consultas críticas

Mejora del 87,77% en tiempo y 88,85% en bloques leídos



Evaluar escalabilidad, CAP y migración a producción

Análisis por módulo, plan 10x, trade-offs CAP y consideraciones técnicas/costo

Reflexión final

No existe un único motor ideal para todos los casos: cada tecnología aporta valor cuando se utiliza en el contexto adecuado. La arquitectura híbrida entre PostgreSQL y MongoDB permitió equilibrar consistencia transaccional con flexibilidad documental, manteniendo una solución viable dentro de las restricciones del free tier.



Decisión por contexto

La selección tecnológica responde al negocio, los patrones de consulta y la criticidad del dato.



Diseño determina rendimiento

Índices, particionamiento y modelado documental impactan directamente la escalabilidad.



Visión arquitectura

Consistencia, disponibilidad, costos, operación y escenarios de falla, evaluados en conjunto.